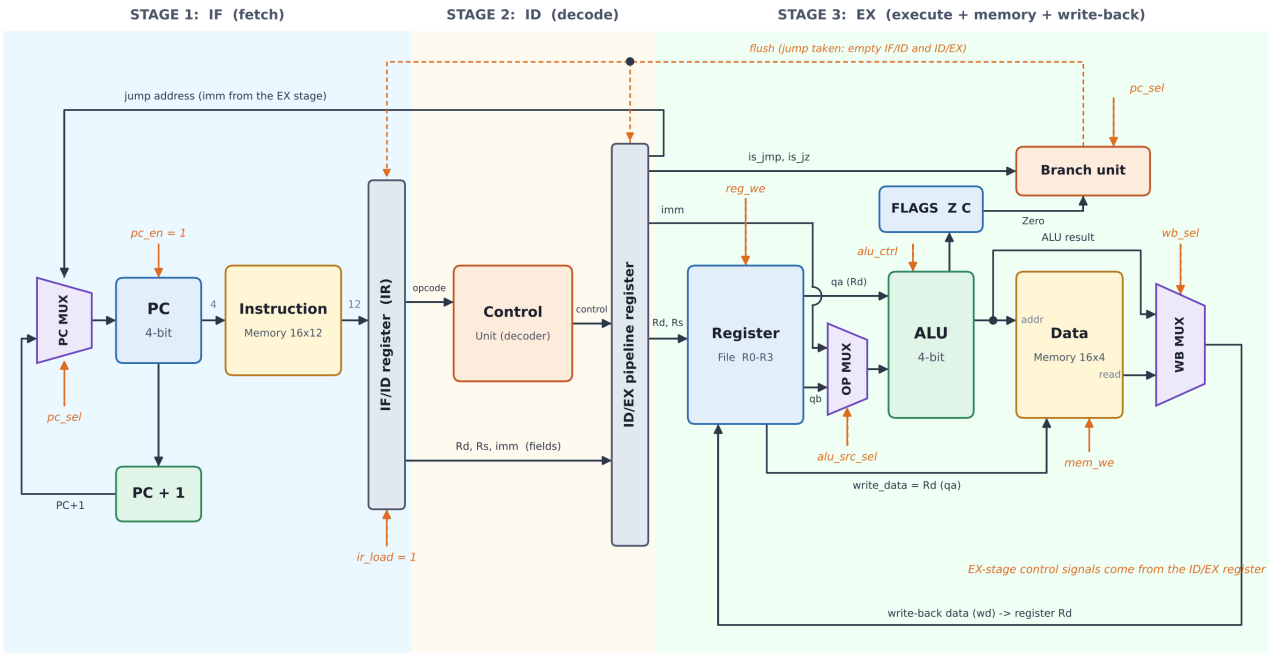
Code walkthrough

cpu_top.v

Top Level (cpu_top)

What it does	Connects every block into the pipelined CPU
Where it works	all stages
Size	263 lines, explained in 21 small parts
Modules in this file	cpu_top
Calls (uses)	mux.v, program_counter.v, instruction_memory.v, instruction_register.v, control_unit.v, pipeline_register.v, register_file.v, alu.v, flag_register.v, data_memory.v
Called by	tb_cpu.v - instance CPU

cpu_top contains almost no logic of its own. It creates one copy of each block and wires them together, stage by stage. Signal names start with id_ or ex_ to show which stage they belong to.



Where every block in cpu_top.v sits in the 3-stage pipeline.

Map of the file

Part	Lines	What this part is
1	1-24	Header comment
2	25-37	Parameter and ports
3	39-42	Wires shared between stages
4	44-50	IF stage: wires
5	52-59	IF stage: PC MUX
6	61-68	IF stage: program counter
7	70-74	IF stage: instruction memory
8	76-92	IF/ID register (the IR)
9	94-107	ID stage: splitting the instruction
10	109-128	ID stage: control unit
11	130-137	ID/EX register: wires
12	139-158	ID/EX register: inputs
13	159-174	ID/EX register: outputs
14	176-185	EX stage: wires
15	187-199	EX stage: register file
16	201-208	EX stage: operand MUX
17	210-219	EX stage: ALU
18	221-232	EX stage: flag register
19	234-242	EX stage: data memory
20	244-251	EX stage: write-back MUX
21	253-263	Debug outputs

How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-24)

cpu_top.v lines 1-24

```
 1  // ======================================================================
 2  // FILE: cpu_top.v
 3  // The complete 4-bit RISC CPU with a 3-STAGE PIPELINE.
 4  // This file only CONNECTS the blocks.
 5  //
 6  //   STAGE 1  IF (fetch)   : PC -> instruction memory -> IF/ID register (IR)
 7  //   STAGE 2  ID (decode)  : control unit decodes -> ID/EX register
 8  //   STAGE 3  EX (execute) : read registers -> operand MUX -> ALU
 9  //                           -> data memory -> write-back MUX -> Rd,
10  //                           update flags, decide jumps
11  //
12  // All three stages work at the same time on three different
13  // instructions, so one instruction can finish every clock.
14  // A taken jump (decided in EX) flushes the 2 younger instructions.
15  //
16  // Instruction format (12 bits):
17  //   [11:8] opcode | [7:6] Rd | [5:4] Rs | [3:0] immediate / address
18  //
19  // CONNECTIONS
20  //   Calls       : mux.v, program_counter.v, instruction_memory.v,
21  //                 instruction_register.v, control_unit.v, pipeline_register.v,
22  //                 register_file.v, alu.v, flag_register.v, data_memory.v
23  //   Called by   : tb_cpu.v - instance CPU
24  // ======================================================================
```

- Summarises the three stages, explains that all of them work at the same time, and shows the 12-bit instruction format.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Parameter and ports (lines 25-37)

cpu_top.v lines 25-37

```
25  module cpu_top #(
26      parameter PROGRAM_FILE = "program.mem"   // program to run
27  )(
28      input         clk,
29      input         rst,
30      // observable debug outputs (waveform viewer or FPGA LEDs)
31      output [3:0]  pc_out,          // fetch address (PC)
32      output [11:0] instr_out,       // instruction in the EXECUTE stage
33      output [3:0]  alu_out,         // ALU result
34      output [3:0]  wb_data_out,     // value written to the destination register
35      output        zero_out,        // Zero flag
36      output        carry_out        // Carry flag
37  );
```

- PROGRAM_FILE is passed down to the instruction memory, so a testbench can choose the program.
- Inputs: only clk and rst.
- Outputs are for watching the CPU (waveform or FPGA LEDs): PC, the instruction in EX, the ALU result, the value being written back, and the two flags.

Part 3 · Wires shared between stages (lines 39-42)

cpu_top.v lines 39-42

```
39      // wires shared between stages
40      wire        pc_sel;            // from the branch unit (EX)
41      wire        flush;             // from the branch unit (EX)
42      wire [3:0]  ex_imm;            // jump address (EX)
```

pc_sel, flush and ex_imm are produced in the EX stage but used back in the IF stage (for jumps), so they are declared first.

Part 4 · IF stage: wires (lines 44-50)

cpu_top.v lines 44-50

```
44      // ================================================================
45      // STAGE 1: IF - FETCH
46      // ================================================================
47      wire [3:0]  pc;                // fetch address
48      wire [3:0]  pc_plus_1 = pc + 4'd1;
49      wire [3:0]  pc_next;
50      wire [11:0] fetched;           // word out of instruction memory
```

- `pc`: the address being fetched.
- `pc_plus_1 = pc + 1`: a small adder that makes the next address.
- `fetched`: the 12-bit word coming out of the instruction memory.

Part 5 · IF stage: PC MUX (lines 52-59)

cpu_top.v lines 52-59

```
52      // PC MUX: next address = PC + 1, or the jump address from EX
53      // CALLS: module mux2 from file mux.v
54      mux2 PC_MUX (
55          .d0  (pc_plus_1),
56          .d1  (ex_imm),
57          .sel (pc_sel),
58          .y   (pc_next)
59      );
```

Chooses the next address: PC + 1 normally (sel = 0), or the jump address from the EX stage when the branch unit sets pc_sel = 1.

→ This part **calls module** `mux2` from file `mux.v`. How that module works is explained in `Explained_mux.pdf`.

Part 6 · IF stage: program counter (lines 61-68)

cpu_top.v lines 61-68

```
61      // CALLS: module program_counter from file program_counter.v
62      program_counter PC (
63          .clk     (clk),
64          .rst     (rst),
65          .pc_en   (1'b1),           // fetch a new instruction every clock
66          .pc_next (pc_next),
67          .pc_out  (pc)
68      );
```

pc_en is tied to 1: the pipeline fetches a new instruction every clock.

→ This part **calls module** `program_counter` from file `program_counter.v`. How that module works is explained in `Explained_program_counter.pdf`.

Part 7 · IF stage: instruction memory (lines 70-74)

cpu_top.v lines 70-74

```
70      // CALLS: module instruction_memory from file instruction_memory.v
71      instruction_memory #(.PROGRAM_FILE(PROGRAM_FILE)) IMEM (
72          .address     (pc),
73          .instruction (fetched)
74      );
```

Address in = pc, instruction out = fetched. The program file name is passed in with #(...).

→ This part **calls module** `instruction_memory` from file `instruction_memory.v`. How that module works is explained in `Explained_instruction_memory.pdf`.

Part 8 · IF/ID register (the IR) (lines 76-92)

cpu_top.v lines 76-92

```
76      // ---------------- IF/ID pipeline register (the IR) ----------------
77      wire [11:0] id_instr;          // instruction now in DECODE
78      wire [3:0]  id_pc;
79      wire        id_valid;
80
81      // CALLS: module instruction_register from file instruction_register.v
82      instruction_register IR (
83          .clk             (clk),
84          .rst             (rst),
85          .ir_load         (1'b1),   // load every clock
86          .flush           (flush),
87          .instruction_in  (fetched),
88          .pc_in           (pc),
89          .instruction_out (id_instr),
90          .pc_out          (id_pc),
91          .valid_out       (id_valid)
92      );
```

- Saves the fetched instruction and its address every clock (ir_load = 1).
- flush comes from the branch unit; on a taken jump the IR becomes empty.
- Outputs get the prefix id_: they are what the ID stage works on.

→ This part **calls module** `instruction_register` from file `instruction_register.v`. How that module works is explained in `Explained_instruction_register.pdf`.

Part 9 · ID stage: splitting the instruction (lines 94-107)

cpu_top.v lines 94-107

```
 94      // ================================================================
 95      // STAGE 2: ID - DECODE
 96      // ================================================================
 97      wire [3:0] id_opcode = id_instr[11:8];
 98      wire [1:0] id_rd     = id_instr[7:6];
 99      wire [1:0] id_rs     = id_instr[5:4];
100      wire [3:0] id_imm    = id_instr[3:0];
101
102      wire [2:0] id_alu_ctrl;
103      wire       id_alu_src_sel, id_wb_sel, id_reg_we, id_mem_we;
104      wire       id_zero_we, id_carry_we, id_is_jmp, id_is_jz;
105
106      wire       ex_is_jmp, ex_is_jz;   // from ID/EX (below)
107      wire       zero_flag, carry_flag; // from the flag register (EX)
```

- The 12 bits are split into fields: opcode [11:8], Rd [7:6], Rs [5:4], imm [3:0].
- Then wires are declared for the decoder outputs (id_...), the jump markers coming back from EX (ex_is_jmp, ex_is_jz) and the saved flags.

Part 10 · ID stage: control unit (lines 109-128)

cpu_top.v lines 109-128

```
109      // CALLS: module control_unit from file control_unit.v
110      control_unit CU (
111          // decode part
112          .opcode      (id_opcode),
113          .alu_ctrl    (id_alu_ctrl),
114          .alu_src_sel (id_alu_src_sel),
115          .wb_sel      (id_wb_sel),
116          .reg_we      (id_reg_we),
117          .mem_we      (id_mem_we),
118          .zero_we     (id_zero_we),
119          .carry_we    (id_carry_we),
120          .is_jmp      (id_is_jmp),
121          .is_jz       (id_is_jz),
122          // branch part (EX stage)
123          .ex_is_jmp   (ex_is_jmp),
124          .ex_is_jz    (ex_is_jz),
125          .zero_flag   (zero_flag),
126          .pc_sel      (pc_sel),
127          .flush       (flush)
128      );
```

- Decoder part: id_opcode in, all id_ control signals out.
- Branch part: works on the EX stage (ex_is_jmp, ex_is_jz, zero_flag in; pc_sel and flush out).

→ This part **calls module** `control_unit` from file `control_unit.v`. How that module works is explained in `Explained_control_unit.pdf`.

Part 11 · ID/EX register: wires (lines 130-137)

cpu_top.v lines 130-137

```
130     // ---------------- ID/EX pipeline register ----------------
131     wire [2:0]  ex_alu_ctrl;
132     wire        ex_alu_src_sel, ex_wb_sel, ex_reg_we, ex_mem_we;
133     wire        ex_zero_we, ex_carry_we;
134     wire [1:0]  ex_rd, ex_rs;
135     wire [3:0]  ex_pc;
136     wire        ex_valid;
137     wire [11:0] ex_instr;
```

Wires for everything the EX stage receives. They all start with ex_.

Part 12 · ID/EX register: inputs (lines 139-158)

cpu_top.v lines 139-158

```
139     // CALLS: module pipeline_register from file pipeline_register.v
140     pipeline_register ID_EX (
141         .clk            (clk),
142         .rst            (rst),
143         .flush          (flush),
144         .alu_ctrl_in    (id_alu_ctrl),
145         .alu_src_sel_in (id_alu_src_sel),
146         .wb_sel_in      (id_wb_sel),
147         .reg_we_in      (id_reg_we),
148         .mem_we_in      (id_mem_we),
149         .zero_we_in     (id_zero_we),
150         .carry_we_in    (id_carry_we),
151         .is_jmp_in      (id_is_jmp),
152         .is_jz_in       (id_is_jz),
153         .rd_in          (id_rd),
154         .rs_in          (id_rs),
155         .imm_in         (id_imm),
156         .pc_in          (id_pc),
157         .valid_in       (id_valid),
158         .instr_in       (id_instr),
```

Every id_ signal (control signals, rd, rs, imm, pc, valid, instruction) goes into the register. flush empties it on a taken jump.

→ This part **calls module** `pipeline_register` from file `pipeline_register.v`. How that module works is explained in `Explained_pipeline_register.pdf`.

Part 13 · ID/EX register: outputs (lines 159-174)

cpu_top.v lines 159-174

```
159          .alu_ctrl       (ex_alu_ctrl),
160          .alu_src_sel    (ex_alu_src_sel),
161          .wb_sel         (ex_wb_sel),
162          .reg_we         (ex_reg_we),
163          .mem_we         (ex_mem_we),
164          .zero_we        (ex_zero_we),
165          .carry_we       (ex_carry_we),
166          .is_jmp         (ex_is_jmp),
167          .is_jz          (ex_is_jz),
168          .rd             (ex_rd),
169          .rs             (ex_rs),
170          .imm            (ex_imm),
171          .pc             (ex_pc),
172          .valid          (ex_valid),
173          .instr          (ex_instr)
174      );
```

One clock later the same values come out as ex_ signals and drive the EX stage.

Part 14 · EX stage: wires (lines 176-185)

cpu_top.v lines 176-185

```
176      // ================================================================
177      // STAGE 3: EX - EXECUTE (operand read, ALU, memory, write-back)
178      // ================================================================
179      wire [3:0] rd_value;           // qa: value of Rd
180      wire [3:0] rs_value;           // qb: value of Rs
181      wire [3:0] alu_b;
182      wire [3:0] alu_result;
183      wire       alu_zero, alu_carry;
184      wire [3:0] mem_data;
185      wire [3:0] wb_data;
```

Wires for the register values (rd_value = qa, rs_value = qb), the ALU input B, the ALU result and flags, the memory output and the write-back value.

Part 15 · EX stage: register file (lines 187-199)

cpu_top.v lines 187-199

```
187      // ---- operand read ----
188      // CALLS: module register_file from file register_file.v
189      register_file RF (
190          .clk (clk),
191          .rst (rst),
192          .ra  (ex_rd),              // port A reads Rd
193          .rb  (ex_rs),              // port B reads Rs
194          .wa  (ex_rd),              // result is written into Rd
195          .wd  (wb_data),
196          .we  (ex_reg_we),
197          .qa  (rd_value),
198          .qb  (rs_value)
199      );
```

- Port A reads Rd, port B reads Rs. Because registers are read here in EX, an instruction always sees the result of the instruction just before it (no data hazard).
- The write port writes wb_data into Rd when ex_reg_we is 1.

→ This part **calls module** `register_file` from file `register_file.v`. How that module works is explained in `Explained_register_file.pdf`.

Part 16 · EX stage: operand MUX (lines 201-208)

cpu_top.v lines 201-208

```
201      // ---- operand MUX: B = Rs value (0) or immediate (1) ----
202      // CALLS: module mux2 from file mux.v
203      mux2 OPERAND_MUX (
204          .d0  (rs_value),
205          .d1  (ex_imm),
206          .sel (ex_alu_src_sel),
207          .y   (alu_b)
208      );
```

ALU input B = Rs value (0) or the immediate (1).

→ This part **calls module** `mux2` from file `mux.v`. How that module works is explained in `Explained_mux.pdf`.

Part 17 · EX stage: ALU (lines 210-219)

cpu_top.v lines 210-219

```
210      // ---- ALU ----
211      // CALLS: module alu from file alu.v
212      alu ALU (
213          .a        (rd_value),
214          .b        (alu_b),
215          .alu_ctrl (ex_alu_ctrl),
216          .result   (alu_result),
217          .carry    (alu_carry),
218          .zero     (alu_zero)
219      );
```

A = Rd value, B = output of the operand MUX, operation = ex_alu_ctrl.

→ This part **calls module** `alu` from file `alu.v`. How that module works is explained in `Explained_alu.pdf`.

Part 18 · EX stage: flag register (lines 221-232)

cpu_top.v lines 221-232

```
221      // ---- flags ----
222      // CALLS: module flag_register from file flag_register.v
223      flag_register FLAGS (
224          .clk        (clk),
225          .rst        (rst),
226          .zero_we    (ex_zero_we),
227          .carry_we   (ex_carry_we),
228          .zero_in    (alu_zero),
229          .carry_in   (alu_carry),
230          .zero_flag  (zero_flag),
231          .carry_flag (carry_flag)
232      );
```

Saves the ALU's Zero and Carry at the clock edge, depending on ex_zero_we and ex_carry_we. zero_flag goes to the branch unit for JZ.

→ This part **calls module** `flag_register` from file `flag_register.v`. How that module works is explained in `Explained_flag_register.pdf`.

Part 19 · EX stage: data memory (lines 234-242)

cpu_top.v lines 234-242

```
234      // ---- memory access: address = ALU result, data = Rd ----
235      // CALLS: module data_memory from file data_memory.v
236      data_memory DMEM (
237          .clk        (clk),
238          .address    (alu_result),
239          .write_data (rd_value),
240          .mem_we     (ex_mem_we),
241          .read_data  (mem_data)
242      );
```

Address = ALU result (the immediate passed through), write data = Rd value, written only when ex_mem_we = 1.

→ This part **calls module** `data_memory` from file `data_memory.v`. How that module works is explained in `Explained_data_memory.pdf`.

Part 20 · EX stage: write-back MUX (lines 244-251)

cpu_top.v lines 244-251

```
244      // ---- write-back MUX: ALU result (0) or memory data (1) ----
245      // CALLS: module mux2 from file mux.v
246      mux2 WB_MUX (
247          .d0  (alu_result),
248          .d1  (mem_data),
249          .sel (ex_wb_sel),
250          .y   (wb_data)
251      );
```

Chooses what goes back into Rd: the ALU result (0) or the memory value (1, for LOAD). Its output wb_data feeds the register file's write port.

→ This part **calls module** `mux2` from file `mux.v`. How that module works is explained in `Explained_mux.pdf`.

Part 21 · Debug outputs (lines 253-263)

cpu_top.v lines 253-263

```
253      // ================================================================
254      // Debug outputs
255      // ================================================================
256      assign pc_out      = pc;
257      assign instr_out   = ex_instr;
258      assign alu_out     = alu_result;
259      assign wb_data_out = wb_data;
260      assign zero_out    = zero_flag;
261      assign carry_out   = carry_flag;
262
263  endmodule
```

Copies internal signals to the output ports so they can be watched in the waveform or on FPGA LEDs.

> **In short:** IF → IF/ID → ID → ID/EX → EX, with pc_sel / flush / jump address going back to IF.